Testing Steps & Expected Output: Implement Client Script & UI Policy (Incident)

Overview

This document outlines the systematic testing procedures and expected outcomes to verify the client-side controls (UI Policies and Client Scripts) implemented on the Incident form and list view in ServiceNow.

Activity 1: Test Mandatory Enforcement (`onSubmit` Client Script & UI Policy Action)

Goal

Verify that an Incident record with **High Impact** cannot be submitted without an **Assigned To** person, and that **Assignment Group** is set as mandatory.

Testing Steps

1. Navigate to **Incident** → **Create New** to open a blank Incident form.
2. Set the **Impact** field to `1 - High`.
3. Set **Assignment group** to `Software`.
4. Leave the **Assigned to** field empty.
5. Click **Submit** at the top right of the form.

Expected Output

- The Incident record **is not submitted** or saved.
- An error message is displayed directly under the **Assigned to** field:*"Assigned To is mandatory for High impact incidents."*
- The **Assignment group** field becomes mandatory (marked with a red asterisk).

Activity 2: Test Successful Record Submission (`onSubmit` Pass Case & `onChange` Client Script)

Goal

Verify that providing all required mandatory values allows successful creation of the Incident, and confirm the auto-setting of the **Urgency** field.

Testing Steps

1. On the same Incident form (`INC0010012` or new), keep **Impact** set to `1 - High`.
2. Observe the **Urgency** field when **Impact** is changed to `1 - High`.
3. Fill in the **Assigned to** field with a valid user (e.g., `Beth Anglin`).
4. Click **Submit**.

Expected Output

- Upon changing **Impact** to `1 - High`, an information banner appears:*"Urgency set to High for High impact incident."*
- **Urgency** automatically sets to `1 - High` and becomes **read-only**.
- The Incident record is saved successfully without any errors or validation warnings.

Activity 3: Test Reverse Condition (`Reverse if false` UI Policy Behavior)

Goal

Verify that changing the **Impact** away from High restores the original field behaviors (clears mandatory status and enables editing).

Testing Steps

1. Open an existing Incident record where **Impact** is currently set to `1 - High` (e.g., `INC0010012`).
2. Update the **Impact** field from `1 - High` to `2 - Medium`.

3. Observe the behavior of the **Urgency** and **Assigned to** fields.
4. Click **Submit** or **Update**.

Expected Output

- **Assigned to** is no longer required/mandatory.
- **Urgency** becomes editable again (no longer read-only).
- The record saves successfully without requiring **Assigned to** to be filled.

Activity 4: Test List Edit Blocking (`onCellEdit` Client Script)

Goal

Verify that users cannot modify the **State** field directly from the Incident list view.

Testing Steps

1. Navigate to **Incident** → **All** or **Open** to open the Incident list view.
2. Locate any Incident record in the list.
3. Double-click on the **State** cell for that record to open the inline list editor.
4. Attempt to select a new State value and click the save checkmark.

Expected Output

- An alert pop-up window appears with the following message:*"State cannot be changed from the list. Please open the incident."*
- The list edit operation is cancelled/rolled back.
- The **State** field value remains unchanged in the list.

Activity 5: Test Form-Based Update (Form View State Change)

Goal

Confirm that State changes are allowed when performed through the standard Incident form view rather than the list view.

Testing Steps

1. Click on the Incident number (e.g., `INC0008111`) from the list to open its form view.
2. Locate the **State** field on the form.
3. Change the **State** field value (e.g., from `New` to `In Progress`).
4. Click **Update** or right-click the header and select **Save**.

Expected Output

- The record updates successfully without triggering the list edit alert.
- The **State** field reflects the updated value (`In Progress`).

Summary Matrix

Activity	Condition / Action	Expected Result	Pass / Fail
1. Missing Assigned To	Impact = 1 - High, Assigned To = empty	Save blocked; error box rendered under Assigned To	**Pass**
2. Successful Save	Impact = 1 - High, Assigned To = Beth Anglin	Save successful; Urgency auto-updated to High & read-only	**Pass**
3. Reverse Condition	Impact changed from 1 - High to 2 - Medium	Assigned To optional; Urgency editable again	**Pass**
4. List Edit Prevention	Double-click State in List View	Alert popped up; change blocked and reverted	**Pass**
5. Form-Based Update	Change State inside Form View and click Update	Record saved successfully with updated State	**Pass**